Voyages scolaires

Concevoir, exposer, valider

Séquence
SQ01

Partie
P04P05P06

Niveau
Mastère ESI

Objectifs

- Concevoir le modèle `Voyage` à partir du besoin
- Exposer les voyages en API JSON
- Rejeter une saisie invalide et expliquer la réponse
- Créer et modifier un voyage par formulaire

Prérequis : parties 1 à 3 — modèle, Eloquent, route, contrôleur, vue Blade.

ATTENTION

Le problème du jour

L'appli mobile du lycée et le secrétariat vont enregistrer des voyages.

Un voyage à **–50 €**, pour **0 élève**, **parti hier** :

qui doit l'empêcher, et à quel endroit ?

Une équipe, un dépôt, trois parts

Temps	Organisation	Production
Concevoir	Seul sur papier, puis en commun	Schéma du voyage
Construire	À trois, un pilote au clavier, rotation	Modèle, API, liste, règle commune sur `main`
Approfondir	Une part chacun, sur sa branche	API, Création, Modification
Restituer	Chacun devant les deux autres	Saisie invalide montrée et expliquée

ACTIVITÉ

Concevoir le voyage

10 min · seul, sur papier · puis mise en commun

À produire : un tableau *information · type · contrainte métier*.

1. Ce qu'un parent consulte pour décider d'une inscription
2. Ce qu'un enseignant saisit pour publier un voyage
3. Pour chaque information : son type, sa règle

Pilote et navigateurs

- **Pilote** : tape, annonce ce qu'il fait
- **Navigateurs** : lisent le support, vérifient chaque ligne, préviennent avant l'erreur
- On change de pilote **à chaque étape**

Étapes : modèle et données de test · API en lecture · API en écriture · liste web.

`$fillable` devient un attribut PHP

```
use Illuminate\Database\Eloquent\Attributes\Fillable;

#[Fillable(['titre', 'destination', 'depart',
            'prix', 'places'])]
class Voyage extends Model {}
```

- Remplace `protected $fillable = [...]`, encore accepté
- `#[...]` : attribut PHP 8, métadonnée lue par Laravel
- Liste blanche de `create()`, **pas une validation**

L'API en une ligne de routes

```
Route::apiResource('voyages', VoyageController::class);
```

Méthode	URL	Action	Succès
GET	`/api/voyages`	`index`	200
POST	`/api/voyages`	`store`	201
GET	`/api/voyages/{voyage}`	`show`	200
PUT/PATCH	`/api/voyages/{voyage}`	`update`	200
DELETE	`/api/voyages/{voyage}`	`destroy`	204

Essayons de casser l'API

```
curl -s -X POST http://127.0.0.1:8000/api/voyages \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{"titre":"Test","destination":"Lune",
       "depart":"2020-01-01","prix":-50,"places":0}'
```

Réponse **201** : le voyage absurde est enregistré. Avec un titre vide : **500**.

Le navigateur ne protège rien

- `required`, `min`, `type="date"` : du **confort** pour l'utilisateur
- `curl`, une appli mobile, un script : ils ne passent pas par le formulaire
- La validation qui protège s'exécute **sur le serveur**

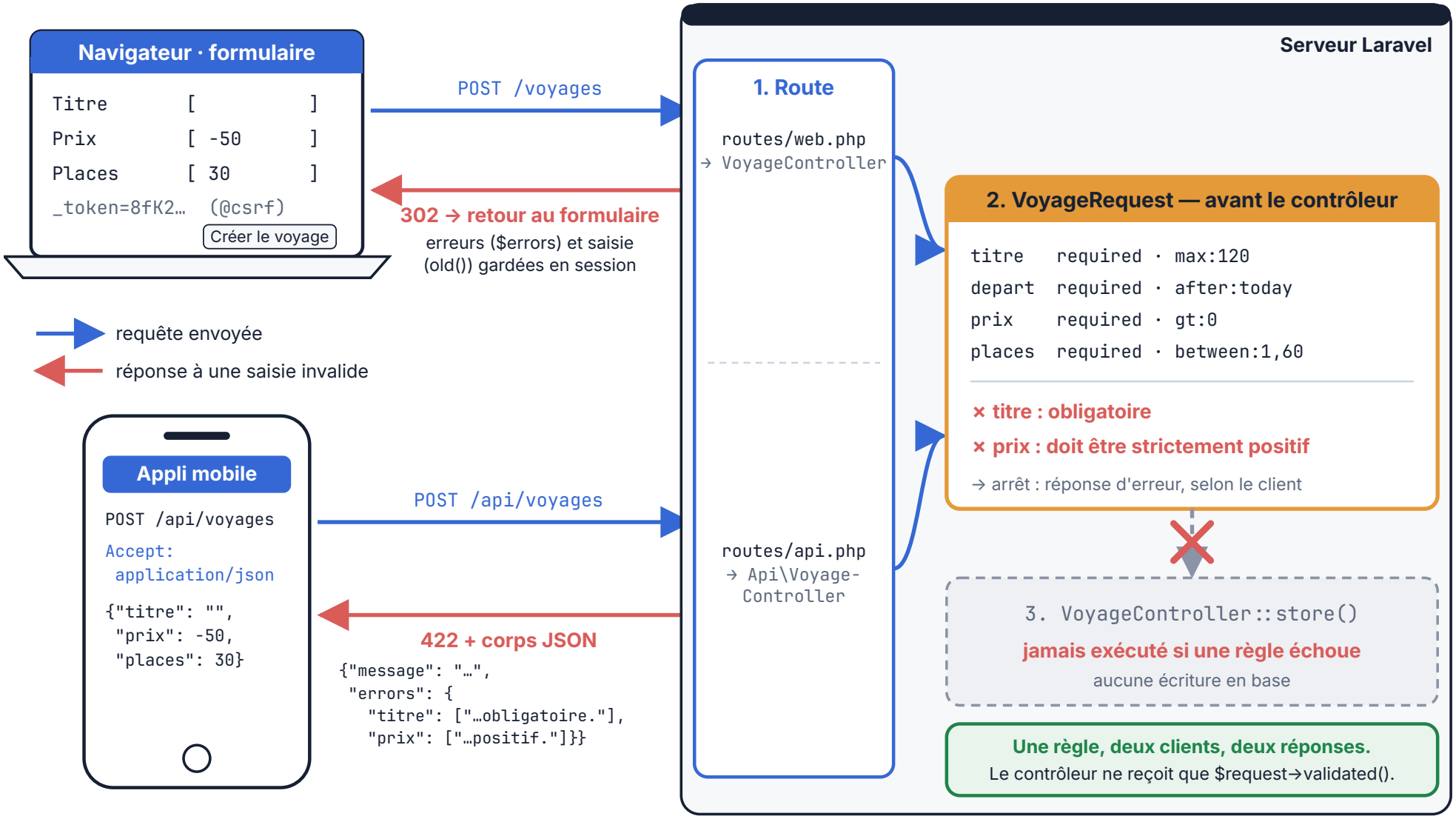
> *Ne jamais faire confiance à une donnée qui vient du client.*

Les règles métier, écrites une fois

```
public function rules(): array
{
    return [
        'titre' => ['required', 'string', 'max:120'],
        'destination' => ['required', 'string', 'max:100'],
        'depart' => ['required', 'date', 'after:today'],
        'prix' => ['required', 'numeric', 'gt:0'],
        'places' => ['required', 'integer', 'between:1,60'],
    ];
}
```

Chaque ligne traduit une contrainte du tableau de conception.

Une règle, deux clients, deux réponses



Le contrôleur ne voit que du valide

```
public function store(VoyageRequest $request)
{
    $voyage = Voyage::create($request->validated());

    return response()->json($voyage, 201);
}
```

- `VoyageRequest` : validation **avant** l'appel
- `validated()` : seulement les champs prévus, un `"id": 999` est ignoré

Les quatre mécanismes d'un formulaire

Élément	Rôle
`@csrf`	Jeton de session ; sans lui, **419**
`@method('PUT')`	Simule PUT ou DELETE, que HTML ne connaît pas
`@error('prix')`	Affiche l'erreur du champ dans `$message`
`old('prix', $voyage->prix)`	Rend la saisie refusée, sinon la valeur enregistrée

Source, sink : où naît une faille

- **Source** : toute donnée contrôlée par le client — formulaire, JSON, URL, en-tête, base
- **Sink** : l'endroit où elle est **interprétée** — HTML, SQL, redirection, commande
- Faille : une source atteint un sink **sans traitement adapté au contexte**

> *Valider en entrée (le métier), échapper en sortie (le contexte).*

Le JSON du `201` n'est pas un sink : c'est le client qui l'affichera qui doit échapper.

Cinq sinks triviaux en Laravel

```
XSS stockée
  {!! $v->titre !!}                  →  {{ $v->titre }}
XSS côté client
  el.innerHTML = v.titre             →  el.textContent = v.titre
Injection SQL
  whereRaw("destination = '$d'") →  where('destination', $d)
Redirection ouverte
  redirect($request->retour)     →  to_route('voyages.index')
Affectation de masse
  #[Unguarded] + all()               →  #[Fillable] + validated()
```

ACTIVITÉ

Une part chacun

Seul · sur sa branche · support A4, parties 5 et 6

À produire : une interface qui refuse une saisie invalide et explique pourquoi.

1. **API** — tester 7 cas, messages en français, README de l'API
2. **Création** — formulaire, erreurs, saisie conservée, message flash
3. **Modification** — formulaire prérempli, suppression, voyage déjà parti

À VOUS

Restitution

Chacun, sur son poste projeté :

1. Montre une saisie invalide et la réponse obtenue
2. Raconte le trajet de la requête
3. Explique une ligne de son code
4. Désigne un sink de sa part et ce qui le protège

Pourquoi le contrôleur n'est-il jamais exécuté quand la saisie est invalide ?

À RETENIR

À retenir

- Le besoin métier devient un schéma, puis des **règles de validation**
- Une Form Request valide **avant** le contrôleur, pour tous les clients
- Navigateur : **302** + erreurs en session · API : **422** + JSON
- `validated()` plutôt que `all()`
- Valider en entrée, **échapper en sortie** : chaque sink a son contexte

Cet après-midi : réunir les trois branches sans rien casser.